

Cahier des Charges : Système de Gestion Universitaire Distribué

1. Présentation du Projet

Titre du Projet : Plateforme de Gestion Universitaire basée sur une Architecture Microservices. Objectif : Concevoir et développer une application web distribuée permettant la gestion des étudiants, des notes, des cours et de la facturation. L'objectif technique est de démontrer l'interopérabilité entre différents langages de programmation (Java, Python, Node.js, .NET) via des protocoles REST et SOAP, orchestrés par une API Gateway sécurisée.

2. Architecture Technique

Le système repose sur une architecture **Microservices Polyglotte** conteneurisée via **Docker**.

2.1. Vue d'ensemble

- **Frontend** : Single Page Application (SPA) en React.
- **Backend** : Ensemble de services autonomes communiquant via HTTP.
- **Base de Données** : MongoDB (NoSQL) pour la persistance des données.
- **Sécurité** : Centralisée via API Gateway et authentification JWT.

2.2. Cartographie des Services

Service	Technologie	Protocole	Port	Rôle
API Gateway	Spring Cloud (Java)	REST/HTTP	8080	Point d'entrée unique, Routage, Filtres de Sécurité.
Auth Service	Spring Boot (Java)	REST	8088	Gestion des utilisateurs, Inscription, Login, Génération de Token JWT.
Student Service	Node.js (Express)	REST	3000	Gestion de l'annuaire des étudiants.
Grade Service	Python (FastAPI)	REST	5000	Gestion des notes (Ajout, Liste, Suppression, Moyenne).
Course Service	Java (JAX-WS)	SOAP	8082	Gestion du catalogue de cours (XML).

Billing Service	.NET Core	SOAP	8083	Gestion de la facturation et des paiements.
Frontend	React + Vite	HTTP	8090	Interface utilisateur Dashboard.
Database	MongoDB	TCP	27017	Stockage des données (Users, Students, Grades).

3. Spécifications Fonctionnelles (Par Rôle)

Le système implémente un **Contrôle d'Accès Basé sur les Rôles (RBAC)** strict.

3.1 Administrateur (ADMIN)

- **Accès Global** : Visibilité complète sur tous les modules.
- **Gestion Étudiants** : Ajouter un nouvel étudiant dans l'annuaire.
- **Gestion Cours** : Ajouter des cours au catalogue (via SOAP).
- **Gestion Facturation** : Créer des factures pour les étudiants (via SOAP).
- **Gestion Notes** : Ajouter ou supprimer n'importe quelle note.

3.2 Enseignant (TEACHER)

- **Gestion Notes** : Ajouter une note à un étudiant. Voir la liste des notes. Supprimer une note (en cas d'erreur).
- **Gestion Cours** : Ajouter de nouveaux cours au catalogue.
- **Annuaire** : Consulter la liste des étudiants.
- **Raccourci** : Depuis la liste des étudiants, cliquer sur un bouton pour accéder directement au formulaire de notation pour cet étudiant.
- **Restriction** : Aucun accès au module de Facturation.

3.3 Étudiant (STUDENT)

- **Consultation** : Voir la liste des étudiants (Lecture seule).
- **Notes** : Voir ses propres notes (Filtrage automatique).
- **Cours** : Consulter le catalogue des cours disponibles.
- **Facturation** : Consulter ses factures en attente et effectuer un paiement (Simulation de paiement SOAP).
- **Restriction** : Impossible d'ajouter ou de modifier des données.

4. Spécifications Techniques Détaillées

4.1. API Gateway & Sécurité

- **Technologie** : Spring Cloud Gateway.
- **Routing Dynamique** : Redirection des requêtes /api/students vers le service Node.js, /api/grades vers Python, etc.
- **Filtre d'Authentification** : Intercepte chaque requête pour vérifier la présence et la validité du Token JWT (Header Authorization: Bearer ...).
- **CORS (Cross-Origin Resource Sharing)** : Configuration stricte autorisant uniquement le Frontend (<http://localhost:8090>).

4.2. Authentification (Auth Service)

- **Persistence** : Stockage des utilisateurs (username, password, role) dans MongoDB.
- **Sécurité** : Génération de tokens JWT signés (HMAC SHA-256) contenant le sub (username) et le role.

4.3. Services Métier

- **Student Service (Node.js)** : API RESTful pour CRUD simple sur la collection students.
- **Grade Service (Python)** : API RESTful avec FastAPI. Utilisation de Pydantic pour la validation des données. Gestion des erreurs robuste (Logs de crash).
- **Course Service (Java SOAP)** : Service Web SOAP exposant un fichier WSDL. Opérations : addCourse, getAllCourses.
- **Billing Service (.NET SOAP)** : Service Web SOAP pour ProcessPayment.

4.4. Frontend (Interface Utilisateur)

- **Framework** : React.js avec Vite.
- **Design** : Tailwind CSS pour une interface responsive et moderne.
- **Logique SOAP** : Construction manuelle des enveloppes XML (<soapenv:Envelope>) côté client pour communiquer avec les services Java et .NET.
- **Gestion d'État** : Utilisation de localStorage pour persister le Token JWT et gestion dynamique de l'affichage selon le rôle décodé du token.

5. Environnement de Déploiement

Le projet est entièrement "Dockerisé" pour garantir la portabilité.

- **Docker Compose** : Un fichier docker-compose.yml unique orchestre le lancement simultané des 7 conteneurs (Services + Base de données).
- **Réseau** : Création d'un réseau privé (soa-network) pour la communication inter-conteneurs.

6. Scénarios de Test (Validation)

1. **Workflow Inscription** : Création d'un compte Admin et vérification de la persistance dans MongoDB (redémarrage du service).
2. **Workflow Enseignant** : Connexion en tant qu'enseignant -> Consultation liste étudiants -> Redirection vers ajout de note -> Vérification de l'ajout dans la liste des notes.
3. **Workflow SOAP** : Ajout d'un cours via le formulaire React -> Vérification de la réponse XML -> Affichage de la carte du cours.
4. **Workflow Sécurité** : Tentative d'accès à l'API Student sans Token -> Réception d'une erreur 401/403 de la Gateway.